

Database

Table of Contents

- Database Documentation
 - Main Tables
 - Integrity Contracts
 - Versioned Migrations
 - Local PostgreSQL and Verification
 - Initial Administrator
 - Backup and Migration Gate

Database Documentation

Production uses normalized PostgreSQL. Versioned SQL files in `06Code/migrations` are the schema change source of truth; `06Code/prisma/schema.prisma` is the application mapping used by Prisma. Database-specific partial/expression indexes, checks, the migration ledger and some referential details intentionally live only in SQL, so an unreviewed Prisma schema push is not an approved deployment mechanism.

Main Tables

Table	Purpose and relevant integrity
<code>branches</code>	Academy locations and active state.
<code>roles</code> , <code>permissions</code> , <code>role_permissions</code>	Application-owned authorization catalog.
<code>users</code> , <code>user_branch_access</code>	Accounts, password-rotation state, role and explicit branch scope.
<code>students</code> , <code>teachers</code>	Academic profiles linked one-to-one with compatible user roles; teacher rate is the current rate.

Table	Purpose and relevant integrity
<code>dance_categories</code> , <code>dance_styles</code> , <code>teacher_styles</code>	Normalized dance catalog and teacher specialties.
<code>class_groups</code>	Branch, level, teacher, style, active state and capacity from 1 to 200.
<code>class_group_enrollments</code>	Student/group membership, status, effective dates, waitlist/history and creator.
<code>class_sessions</code>	Schedule, lifecycle, cancellation/completion actor and attendance-finalization metadata.
<code>student_attendance_records</code>	One versioned attendance result per student/session with recorder, updater and correction reason.
<code>teacher_attendance_records</code>	Session check-in/out, historical hourly-rate snapshot and capped payable minutes.
<code>absence_justifications</code>	Separate pending/approved/rejected review; an approval does not rewrite the physical absence.
<code>scholarship_rules</code> , <code>scholarship_evaluations</code>	Attendance/minimum-session rule, exact evaluation window and director evidence.
<code>level_promotion_evaluations</code>	B1-to-B2 decision/window evidence and evaluator.
<code>enrollment_requests</code>	Lead pipeline, follow-up, loss notes and converted student link.
<code>academy_events</code>	Events/shows and recognized event income.
<code>student_payments</code>	Immutable posted ledger: charges, historical accounting branch, statuses, due/paid dates and linked negative reversals.
<code>sessions</code>	Server-side token hashes, expiry and revocation.
<code>audit_logs</code>	Actor, action, entity, entity id, sanitized metadata and timestamp.
<code>schema_migrations</code>	Migration filename, SHA-256 checksum and application timestamp. Managed by the runner.

Integrity Contracts

- Student and teacher identities are separated from academic profiles; a user cannot be linked to two profiles of the same type.

- Branch and level must match between a student and a class group. `active` and `trial` consume capacity; overflow is waitlisted.
- A group capacity cannot be reduced below its current occupied seats.
- A student/session attendance row is unique. Final attendance requires the exact historical roster; subsequent changes are versioned and require an authorized director plus a reason.
- Class sessions must start as `scheduled`, last at most six hours and cannot overlap another active session for the same group or teacher. Completed/cancelled sessions are immutable.
- A teacher/session pair cannot produce duplicate payable shifts. Closed shifts store the rate applied and payable minutes so later rate changes do not rewrite history.
- A non-cancelled charge is unique by student, month and normalized concept. Its accounting branch is derived when created and retained through a later student transfer. A paid or cancelled entry is immutable; a paid correction is a linked negative reversal.
- Passwords are one-way hashes, session tokens are stored only as hashes, and first-login accounts require a password change.

Versioned Migrations

File	Scope
<code>001_initial_schema.sql</code>	Core normalized schema.
<code>002_account_login_policy.sql</code>	First-login/password-change state.
<code>003_unique_profile_user_links.sql</code>	Unique profile-to-user links.
<code>004_reports_events_profile.sql</code>	Events, finance/report fields and profile photo.
<code>005_academic_enrollment_attendance_integrity.sql</code>	Enrollment history, capacity, roster and attendance audit fields.
<code>006_lead_pipeline_integrity.sql</code>	Controlled commercial pipeline and follow-up.
<code>007_session_lifecycle_audit.sql</code>	Completion, cancellation and attendance finalization.
<code>008_teacher_payroll_integrity.sql</code>	Rate snapshots, payable minutes and duplicate protection.

File	Scope
<code>009_financial_ledger_audit.sql</code>	Auditable charges and reversals.
<code>010_reference_catalog_and_data_quality.sql</code>	Reference seed, non-null/default hardening and quality checks.
<code>011_academic_decision_concurrency.sql</code>	Session/finalization coherence, evaluation windows and partial-unique approved scholarship/promotion decisions.
<code>012_historical_group_reenrollment.sql</code>	Multiple historical enrollment episodes with one partial-unique current student/group link.

The runner obtains PostgreSQL advisory lock `30716`, creates `schema_migrations`, checks immutability by checksum and applies each pending file in its own transaction. A failed file rolls back and is not recorded.

Migration 011 adds its session coherence CHECK as `NOT VALID` so legacy rows do not block rollout, while PostgreSQL still enforces it for new/updated rows; legacy inconsistencies must be reconciled before a later explicit validation. Migrations 001–012 and their restrictions were also exercised from an empty PostgreSQL 16 database.

```
cd 06Code
npm run db:generate
npm run db:migrate:status
npm run db:migrate
npm run db:migrate:status
```

Never edit an already-applied migration. Add the next numbered SQL file and review both its forward compatibility and Prisma mapping.

Local PostgreSQL and Verification

The Docker database binds only to loopback. Set `POSTGRES_PASSWORD` and `DATABASE_URL` in an ignored local environment file before starting it.

```
cd 06Code
npm run db:local:up
npm run db:migrate
npm run db:smoke:operational
npm run db:local:down
```

`db:smoke:operational` validates operational model reads/writes inside a transaction and rolls the transaction back. `db:smoke:prisma` remains a lighter connectivity/mapping check.

Initial Administrator

After migrations, use `npm run db:bootstrap-admin` with the four explicit `BOOTSTRAP_ADMIN_*` variables documented in the root README. The command refuses to run if an active Admin already exists, requires a strong one-time password, writes an audit entry and never prints the password. Remove those variables immediately after use.

Demo/role seeds require development, a localhost database and `ALLOW_LOCAL_DEMO_SEEDS=true`. They are not a production bootstrap path.

Backup and Migration Gate

Before every production migration:

1. Take and verify an RDS snapshot/PITR state.
2. Run `db:migrate:status` against the target and review pending filenames.
3. Deploy one migration runner instance; the advisory lock prevents concurrent execution.
4. Run both smoke checks and critical API probes after migration.
5. Keep the snapshot until functional, financial and attendance reconciliation passes.